

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DATA STRUCTURES AND ALGORITHMS - CO2003

ASSIGNMENT 1

BKMove: BUS ROUTE MANAGEMENT AND JOURNEY SEARCH

Ho Chi Minh City, 09/2026

ASSIGNMENT'S SPECIFICATION

Version 1.0

1 Assignment's outcome

After completing this major assignment, students will be able to:

- Implement and manipulate linear and circular linked-list data structures.
- Apply abstract data structures to model a small real-world application.
- Implement the generic Quick Sort algorithm and use a custom comparator to define application-specific ordering.
- Combine data structures and algorithms to solve direct-route and bounded one-transfer journey-search problems.

2 Introduction

Public transportation applications must answer questions that appear simple to users but require careful data organization internally. A bus route has an ordered sequence of stops in the outbound direction and another ordered sequence in the inbound direction. These two sequences may not be exact reverses of each other. A stop may appear in both directions, while some stops may appear in only one direction.

In this assignment, students will build a simplified bus-route application named **BKMove**. The assignment begins with fundamental linked-list implementations and gradually integrates them into a route-management and journey-search system. The system does not attempt to model a complete city-wide graph. Instead, it focuses on journeys that are either direct or use at most one transfer.

The assignment is organized into five questions:

- **Question 1 – Singly Linked List:** Implement the required operations of `SLinkedList<T>`.
- **Question 2 – Circular Linked List:** Implement `CircularLinkedList<T>` while maintaining a circular topology.
- **Question 3 – Bus Route:** Build a route abstraction on top of the linked-list structures and support direction-aware queries.
- **Question 4 – Quick Sort:** Implement a generic Quick Sort with comparator and pivot-selection support.

- **Question 5 – BKMove Search:** Search and rank direct journeys and journeys with at most one transfer.

Each question is worth **2 points**, for a total of **10 points**.

The questions are designed to be completed incrementally. Questions 1 and 2 provide the data-structure foundation for Question 3. Question 4 provides the sorting algorithm used by Question 5. Question 5 then combines the previous components into the final BKMove application.

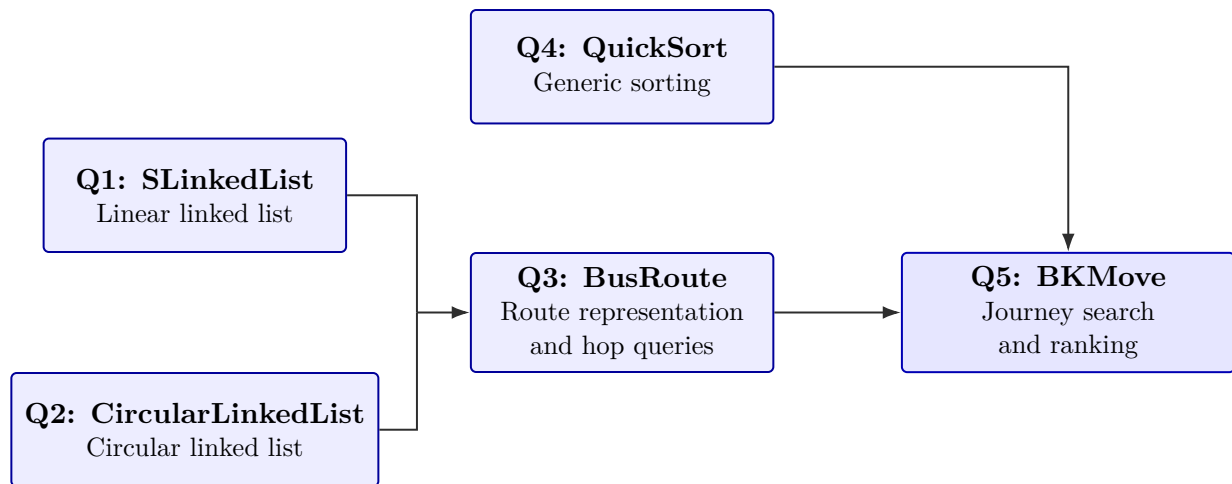


Figure 1: Dependency flow among the five questions in the BKMove assignment.

3 Background and Application Model

3.1 Singly Linked List

A singly linked list stores elements in nodes connected by a **next** pointer. Unlike an array, the nodes are not required to occupy contiguous memory. Insertion and removal therefore depend on manipulating links between nodes rather than shifting a block of elements.

The starter code provides the node definition, the list metadata, constructors, destructors, iterators, and several utility methods. Students are responsible for implementing the core list operations required by the `IList<T>` interface.

3.2 Circular Linked List

A circular linked list is a linked list in which the last node links back to the first node instead of terminating at `nullptr`. Therefore, traversal logic that is correct for an ordinary singly linked list may not be correct for a circular linked list.

The implementation in this assignment uses explicit `head`, `tail`, and `count` fields. Students must maintain a valid circular structure after every insertion, removal, and clear operation.

3.3 Bus Routes and Directions

Each bus route has two logical directions:

- **OUTBOUND**: travel from the first terminal to the second terminal.
- **INBOUND**: travel from the second terminal back to the first terminal.

The outbound and inbound stop sequences are not assumed to be exact reverses. For example, the following two sequences are valid:

OUTBOUND: A -> B -> C -> D -> E

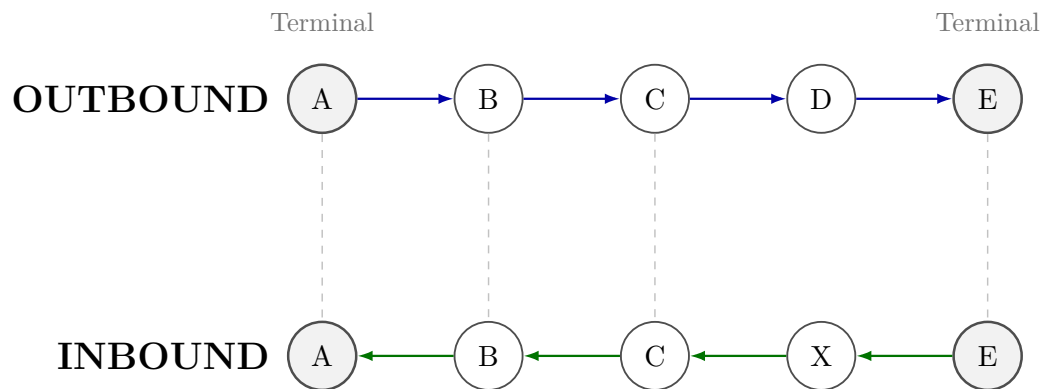
INBOUND: E -> X -> C -> B -> A

A `BusRoute` stores one complete round trip using a `CircularLinkedList<BusStop>` and must expose both directions through the public API. The internal representation is fixed by the starter code; students must determine how to correctly map the logical directional views to that representation.

3.4 Quick Sort and Ranking

The BKMov application can produce multiple valid route or journey candidates. These candidates must be returned in a deterministic ranked order. Students will first implement a generic **Quick Sort** and then reuse it in Question 5 with application-specific comparators.

Quick Sort is a divide-and-conquer sorting algorithm. It repeatedly chooses a pivot, partitions the current range around that pivot, and recursively sorts the resulting subranges. The starter code provides one level of decomposition through `sort`, `quickSort`, and `partition`; students are free to add private helpers if needed.



The two directions may share stops, but need not be exact reverses.

Figure 2: Outbound and inbound directions of a bus route.

4 Requirements

Students are required to complete the five questions below. The public APIs in the provided header files must not be changed. Public tests are included in the starter package and form part of the executable specification of the assignment, especially for boundary cases and special-case behavior.

4.1 Question 1 – Class `SLinkedList<T>` (2 points)

The `SLinkedList<T>` class implements the provided `IList<T>` interface. The class uses a pointer-based linked structure and stores its current number of elements in `count`.

The relevant class structure is shown below. Provided methods are omitted for brevity.

```

1 template<class T>
2 class SLinkedList : public IList<T> {
3 public:
4     class Iterator;
5     class Node;
6
7 protected:
8     Node* head;
9     Node* tail;
10    int count;
11    bool (*itemEqual)(T& lhs, T& rhs);
12    void (*deleteUserData)(SLinkedList<T>*);
13
14 public:

```

```
15     void add(T e) override;
16     void add(int index, T e) override;
17     T removeAt(int index) override;
18     bool removeItem(T item,
19                     void (*removeItemData)(T) = 0) override;
20     void clear() override;
21     T& get(int index) override;
22     int indexOf(T item) override;
23
24     // Other methods are provided in the starter code.
25 };
```

Students must implement the following methods:

1. **void add(T e);**

- Appends **e** to the end of the list.
- The required append operation must run in $O(1)$ time.

2. **void add(int index, T e);**

- Inserts **e** at the requested position.
- Valid and invalid boundary behavior is demonstrated by the public tests.

3. **T removeAt(int index);**

- Removes the element at the requested position and returns the removed value.
- The list metadata and links must remain correct after removing the first, last, middle, or only element.

4. **bool removeItem(T item, void (*removeItemData)(T) = 0);**

- Removes the first matching item according to the equality rule used by the class.
- If **removeItemData** is provided, it must be applied to the removed value as demonstrated by the public tests.

5. **void clear();**

- Removes all data nodes and restores an empty reusable list.
- Dynamically allocated node memory must be released correctly.

6. **T& get(int index);**

- Returns a reference to the value at the requested position.

7. **int indexOf(T item);**

- Returns the position of the first matching element, or the not-found value demonstrated by the public tests.
- Matching must respect the optional `itemEqual` callback when it is provided.

Students must not replace the required linked-list representation with an STL sequence container.

4.2 Question 2 – Class `CircularLinkedList<T>` (2 points)

The `CircularLinkedList<T>` class also implements `IList<T>`, but stores its elements in a circular node structure.

```
1 template<class T>
2 class CircularLinkedList : public IList<T> {
3 public:
4     class Node;
5
6 protected:
7     Node* head;
8     Node* tail;
9     int count;
10    bool (*itemEqual)(T& lhs, T& rhs);
11    void (*deleteUserData)(CircularLinkedList<T>*);
12
13 public:
14    void add(T e) override;
15    void add(int index, T e) override;
16    T removeAt(int index) override;
17    bool removeItem(T item,
18                    void (*removeItemData)(T) = 0) override;
19    void clear() override;
20    T& get(int index) override;
21    int indexOf(T item) override;
22
23    // Other methods are provided in the starter code.
24 };
```

Students must implement the same seven core operations as in Question 1, but every operation must preserve a valid circular topology. In particular, implementations must correctly handle transitions among an empty list, a one-node list, and a list containing multiple nodes.

The append operation `add(T e)` must run in $O(1)$ time. Students must not replace the circular linked-list representation with an STL sequence container.

The public tests cover insertion, removal, invalid indices, equality callbacks, copy behavior, clearing, and reusing the list after it becomes empty.

4.3 Question 3 – Class `BusRoute` (2 points)

The `BusRoute` class represents one complete bus service round trip. Its public interface and internal data members are fixed by the starter code.

```
1 class BusRoute {
2 private:
3     string routeId;
4     CircularLinkedList<BusStop> stops;
5     int outboundCount;
6     bool built;
7
8 private:
9     int physicalIndex(int index, Direction direction);
10
11 public:
12     explicit BusRoute(string routeId);
13
14     string getId() const;
15     bool isBuilt() const;
16
17     void build(SLinkedList<BusStop>& outbound,
18               SLinkedList<BusStop>& inbound);
19     int getStopCount(Direction direction);
20     BusStop& getStop(int index, Direction direction);
21     int getHopCount(string fromStopId,
22                     string toStopId,
23                     Direction direction);
24 };
```

The input to `build` is guaranteed to be valid. Each directional list contains at least two stops. The final stop of `outbound` is the first stop of `inbound`, and the final stop of `inbound` is the first stop of `outbound`. Within one direction, stop IDs are unique. A stop may still appear in both directions because the two directions represent different visits of the service.

Students must implement the following methods:

1. `void build(SLinkedList<BusStop>& outbound, SLinkedList<BusStop>& inbound);`
 - Builds the route from the two logical directional sequences.
 - The input lists must remain unchanged.
 - Calling build again replaces the previously built route.
2. `int getStopCount(Direction direction);`
 - Returns the number of stops visible in the selected logical direction.
3. `int physicalIndex(int index, Direction direction);`
 - Maps a direction-specific logical index to the corresponding position in the internal circular storage.
 - This method is private and is provided as one level of scaffolding for the route abstraction.
4. `BusStop& getStop(int index, Direction direction);`
 - Returns the stop at the requested logical position in the selected direction.
 - Boundary behavior is specified by the public tests.
5. `int getHopCount(string fromStopId, string toStopId, Direction direction);`
 - Returns the number of route segments traveled from fromStopId to toStopId while respecting the selected direction and stop order.
 - If the requested directional journey is not valid, the method returns the failure value specified by the public tests.

The implementation of `BusRoute` must use its required `CircularLinkedList<BusStop>` representation. Auxiliary STL sequence containers are not allowed in `BusRoute.cpp`.

4.4 Question 4 – Class `QuickSort<T>` (2 points)

The `QuickSort<T>` class implements the provided `ISort<T>` interface.

```
1 template<class T>
2 class QuickSort : public ISort<T> {
3 private:
4     int (*pivotSelection)(T*, int);
5
6 public:
7     QuickSort(int (*pivotSelection)(T*, int) = 0);
8
9     void sort(T array[], int size,
```

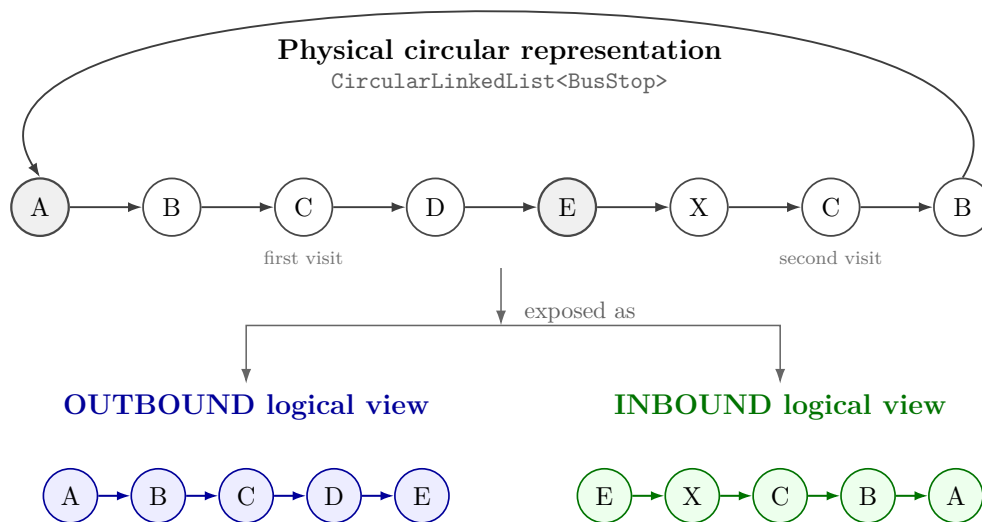


Figure 3: A single circular representation of a bus route and its two logical directional views.

```

10         int (*comparator)(T&, T&) = 0) override;
11
12 private:
13     void quickSort(T array[], int left, int right,
14                   int (*comparator)(T&, T&) = 0);
15
16     int partition(T array[], int left, int right,
17                  int (*comparator)(T&, T&) = 0);
18 };

```

Students must implement all three methods shown above. The provided decomposition is intentionally limited to one level; students may add private helpers if they find them useful.

- With no custom comparator, the class must sort elements in ascending order using the ordinary < and > relations.
- With a comparator, the result must follow the ordering convention demonstrated by the public tests.
- If a custom `pivotSelection` callback is provided, the implementation must use it when choosing pivots.
- The implementation must correctly handle empty arrays, one-element arrays, duplicate values, already sorted input, reverse-sorted input, and all-equal input.
- Standard sorting functions such as `std::sort`, `std::stable_sort`, and `qsort` are not allowed.

4.5 Question 5 – Class BKMove (2 points)

The BKMove class stores a collection of bus routes and provides the final search functionality of the assignment.

```
1 class BKMove {
2 private:
3     vector<BusRoute*> routes;
4
5 public:
6     BKMove() = default;
7
8     void addRoute(BusRoute* route);
9     int getRouteCount() const;
10    BusRoute* getRoute(int index);
11
12    vector<RouteResult> findDirectRoutes(
13        string fromStopId,
14        string toStopId
15    );
16
17    vector<JourneyResult> findJourneys(
18        string fromStopId,
19        string toStopId
20    );
21 };
```

The starter code already implements `addRoute`, `getRouteCount`, and `getRoute`. Students must implement the two search methods.

4.5.1 Direct Route Search

The method

```
vector<RouteResult> findDirectRoutes(string fromStopId, string toStopId)
```

must find every route-direction pair that can travel from the origin to the destination without a transfer. A route that supports both directions produces two distinct candidates.

The returned results must be ranked according to the following priority:

1. Smaller `hopCount` first.

2. If tied, lexicographically smaller `routeId` first.
3. If still tied, OUTBOUND before INBOUND.

The ranking must use the `QuickSort` implementation from Question 4.

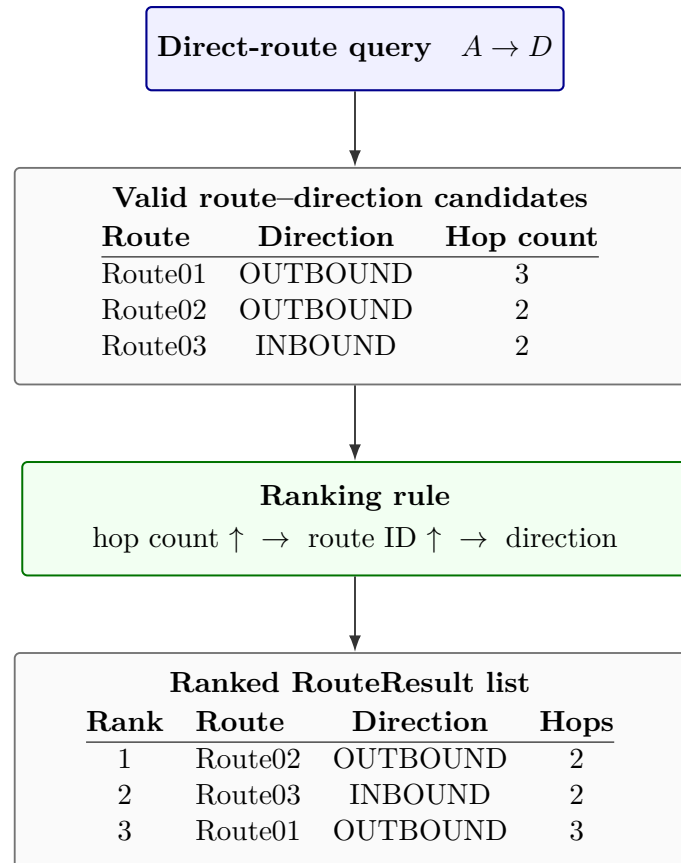


Figure 4: Direct-route discovery and ranking for a query from stop A to stop D.

4.5.2 Journey Search with At Most One Transfer

The method

```
vector<JourneyResult> findJourneys(string fromStopId, string toStopId)
```

must return all valid journeys that use either zero transfers or exactly one transfer.

A one-transfer journey consists of two different route IDs. The transfer stop must occur strictly after the origin on the first leg and strictly before the destination on the second leg; therefore, each leg contains at least one hop. If multiple transfer stops create valid journeys, each valid choice is represented by a separate `JourneyResult`.

For a one-transfer journey, `totalHops` is the sum of the hop counts of the two legs.

The returned journeys must be ranked according to the following priority:

1. Fewer transfers first.
2. Smaller `totalHops`.
3. Lexicographically smaller `firstRouteId`.
4. OUTBOUND before INBOUND for `firstDirection`.
5. Lexicographically smaller `transferStopId`.
6. Lexicographically smaller `secondRouteId`.
7. OUTBOUND before INBOUND for `secondDirection`.

Direct journeys must be included in the same result vector and therefore appear before any one-transfer journey according to the first ranking criterion. If no valid journey exists, an empty vector is returned.

Students may use `std::vector` for the `BKMove` route collection and result collections. However, they must not copy the route topology into an auxiliary STL container of `BusStop`. Search logic must use the public `BusRoute` abstraction. Standard sorting functions are not allowed; ranking must use `QuickSort` from Question 4.

This question is intentionally bounded to at most one transfer. General graph search is not required in this assignment.

5 Public Tests and Checkpoints

The starter package contains public tests grouped by question. Students are expected to use these tests incrementally while implementing the assignment.

Run all public tests for one question using:

```
make q1, make q2, make q3, make q4, or make q5
```

Run all public tests using:

```
make all
```

Each question contains several named test groups. To run one group while debugging, use:

```
make q3 TEST=4
```

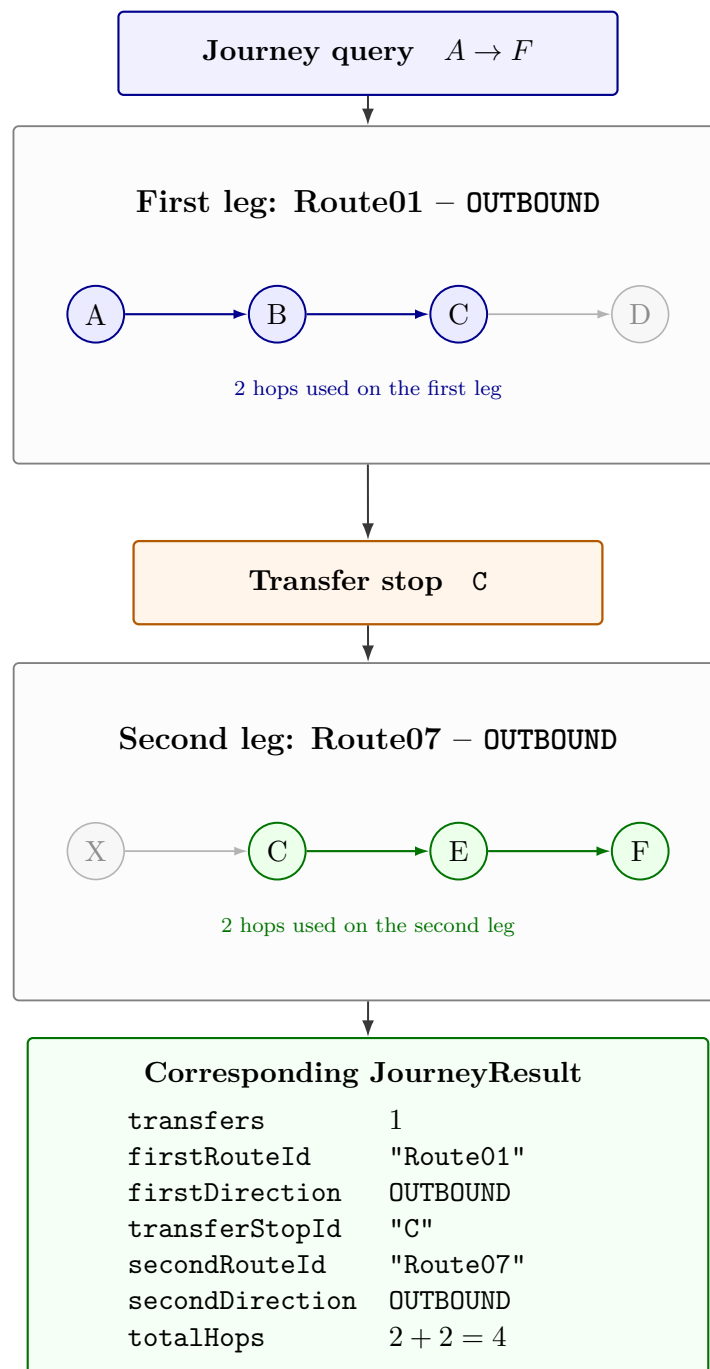


Figure 5: Example of a one-transfer journey and its corresponding JourneyResult.

The public test groups are summarized below:

- **Q1 – SLinkedList:** 7 public groups covering empty/single-item behavior, insertion, indices, removal, equality/callback behavior, copy/assignment, and clear/reuse.
- **Q2 – CircularLinkedList:** 7 public groups covering analogous operations under circular-list behavior.

- **Q3 – BusRoute:** 7 public groups covering initial state, building, directional views, index boundaries, hop counts, minimal routes, and rebuilding.
- **Q4 – QuickSort:** 5 public groups covering base cases, default sorting, difficult input orders, custom comparator, and custom pivot selection.
- **Q5 – BKMove:** 10 public groups covering direct search, direct ranking, one-transfer discovery, transfer validity, all ranking tie-breaks, and no-result cases.

Public tests are intentionally not exhaustive. Hidden tests may use additional values and operation sequences while following the same contracts. Structural and complexity requirements may also be checked separately.

6 Instructions

To complete this assignment, students must:

1. Read this specification and inspect the provided starter code and public tests before implementing the questions.
2. Download and extract the starter package. The important files are:
 - `IList.h`, `ISort.h`, `main.h`: provided interfaces and shared application types.
 - `SLinkedList.h`: Question 1.
 - `CircularLinkedList.h`: Question 2.
 - `BusRoute.h`, `BusRoute.cpp`: Question 3. The header is fixed; students implement the required methods in `BusRoute.cpp`.
 - `QuickSort.h`: Question 4.
 - `BKMove.h`, `BKMove.cpp`: Question 5. The header is fixed; students implement the required methods in `BKMove.cpp`.
 - `tests/`: public tests.
 - `Makefile`: compilation, public checkpoints, and basic source-rule checks.
3. Students must not change the public APIs in `IList.h`, `ISort.h`, `BusRoute.h`, `BKMove.h`, or `main.h`. The grading system may replace these files with the original versions.
4. Students may add private helper methods or additional internal classes when useful, provided that the required public APIs and representations are preserved.
5. The following source restrictions apply:
 - STL sequence containers such as `vector`, `list`, `forward_list`, and `deque` must not be used to implement `SLinkedList`, `CircularLinkedList`, or `BusRoute`.

- Standard sorting functions must not be used for Questions 4 or 5.
 - `BKMove` may use `std::vector` for route pointers and result objects, but must not copy route stops into an auxiliary STL container of `BusStop`.
 - Question 5 must use the `QuickSort` implementation from Question 4 for ranking.
6. Students must correctly deallocate dynamically allocated node memory. Hidden grading may include long operation sequences and memory-related checks.
7. The program will be graded on a UNIX environment using a C++17 compiler. Students should resolve any compilation or runtime differences before the final submission.

7 Submission

Students must submit only the following five files:

- `SLinkedList.h`
- `CircularLinkedList.h`
- `BusRoute.cpp`
- `QuickSort.h`
- `BKMove.cpp`

The deadline and submission link will be announced on the course LMS. Students may submit multiple times before the deadline; only the final valid submission will be graded. Students are responsible for submitting early enough to handle compilation or platform issues.

8 Other regulations

- Students must complete this assignment independently and protect their source code from unauthorized copying.
- Submitted code may be screened for structural similarity, prohibited constructs, and unusual implementation patterns. A flagged submission may be selected for a short code-explanation or code-modification check. A flag alone does not automatically determine a penalty.
- Students may be asked to explain or make a small modification to any part of their submitted implementation.
- Hidden testcases are not released after grading. Instructors may instead provide information about testcase categories or common failure patterns.



- Any decision made by the instructors in charge of this assignment is final according to the course and university regulations.

9 Changelog

—————END—————